

22CS22013 《数据库系统》实验报告

姓名：顾雨航

学号：2023112451

学院：未来技术学院

专业：人工智能

班级：23R0361

日期：2026.06.09

教师评语

[!WARNING]

此处由教师填写评语，请不要改动

声明

本人承诺该实验全部由本人独立完成，没有抄袭他人代码。若被证实本人存在抄袭现象或为他人抄袭提供帮助，本人愿意承担全部责任（包括但不限于扣分、取消考试资格、上报学院/学部等）。

目录

- 声明
- 实验1：缓冲池管理器实现
 - 任务1：缓冲池页面替换策略实现
 - 任务2：缓冲池管理器实现
- 实验2：记录管理器实现
 - 任务1：记录操作实现
 - 任务2：记录迭代器实现
- 实验3：数据定义的实现
 - 任务1：元数据管理实现
- 实验4：数据操纵的实现
 - 任务1：顺序扫描算子的实现
 - 任务2：投影算子的实现
 - 任务3：嵌套循环连接算子的实现
 - 任务4：修改算子的实现
 - 任务5：删除算子的实现
- 总结

实验1：缓冲池管理器实现

任务1：缓冲池页面替换策略实现

(1) 代码分析

本任务补全 `LRUReplacer` 类，实现最近最少使用页面替换策略。缓冲池中的一个内存槽位称为 `frame`，一个 `frame` 可以缓存一个磁盘页。当缓冲池没有空闲 `frame` 时，`BufferPoolManager` 需要调用替换器选择一个可以淘汰的 `frame`。

`LRUReplacer` 继承自抽象基类 `Replacer`。`Replacer` 只规定 `victim`、`pin`、`unpin`、`Size` 等接口，不关心具体替换算法。这样缓冲池管理器只依赖统一接口，不需要知道底层使用 LRU 还是其他策略。

`LRUReplacer` 的核心成员变量如下：

- `latch_`：互斥锁，用来保护 `LRUlist_` 和 `LRUhash_`，避免并发访问时链表与哈希表状态不一致。
- `LRUlist_`：保存当前可以被淘汰的 `frame_id`。本次实现中链表头部表示最久未使用的 `frame`，链表尾部表示最近被取消固定的 `frame`。
- `LRUhash_`：保存 `frame_id` -> 链表迭代器 的映射，用于快速判断某个 `frame` 是否在 LRU 中，并支持 $O(1)$ 删除。
- `max_size_`：替换器最多管理的 `frame` 数量，通常等于缓冲池大小。

`LRUlist_` 负责维护先后顺序，`LRUhash_` 负责快速定位。二者保存的是同一批可淘汰 `frame`，插入和删除时必须同步更新。

(2) `LRUReplacer::victim` 函数的实现

`victim` 的作用是从当前可淘汰集合中选择一个牺牲 `frame`。LRU 策略要求淘汰最久未使用的 `frame`，因此本实现取 `LRUlist_` 的头部元素。

```
bool LRUReplacer::victim(frame_id_t* frame_id) {
    std::scoped_lock lock{latch_};
    if (frame_id == nullptr || LRUlist_.empty()) {
        return false;
    }
    *frame_id = LRUlist_.front();
    LRUhash_.erase(*frame_id);
    LRUlist_.pop_front();
    return true;
}
```

实现步骤为：先加锁保护内部结构；如果链表为空，说明没有可淘汰 `frame`，返回 `false`；否则将链表头部 `frame` 赋给 `*frame_id`，并同步从哈希表和链表中删除该 `frame`。

(3) `LRUReplacer::pin` 函数的实现

`pin` 表示某个页面正在被使用，不能被替换算法淘汰。因此如果该 `frame` 当前在 LRU 可淘汰集合中，需要把它移除。

```
void LRUReplacer::pin(frame_id_t frame_id) {
    std::scoped_lock lock{latch_};
    auto it = LRUhash_.find(frame_id);
    if (it == LRUhash_.end()) {
        return;
    }
}
```

```
    }  
    LRUlist_.erase(it->second);  
    LRUhash_.erase(it);  
}
```

这里通过 `LRUhash_` 判断 `frame` 是否存在，并直接得到链表迭代器。若不存在，说明该 `frame` 本来就不可淘汰，直接返回；若存在，则同时删除链表节点和哈希表项。

(4) `LRUReplacer::unpin` 函数的实现

`unpin` 表示页面暂时不再被使用。当缓冲池中某页的 `pin_count` 变为 0 时，缓冲池管理器调用该函数，将对应 `frame` 加入可淘汰集合。

```
void LRUReplacer::unpin(frame_id_t frame_id) {  
    std::scoped_lock lock{latch_};  
    if (frame_id < 0 || static_cast<size_t>(frame_id) >= max_size_ ||  
        LRUhash_.count(frame_id) != 0) {  
        return;  
    }  
    LRUlist_.push_back(frame_id);  
    auto it = LRUlist_.end();  
    --it;  
    LRUhash_[frame_id] = it;  
}
```

实现中先过滤非法编号和重复加入的 `frame`，然后将 `frame` 插入链表尾部，表示它是最近一次进入可淘汰集合的页面。插入后记录其迭代器到哈希表，便于后续 `pin` 或 `victim` 时维护一致性。

(5) 自己声明的函数

本任务没有额外声明新的公开函数。实现中主要使用 C++ 标准库 `std::list`、`std::unordered_map` 和 `std::scoped_lock` 辅助完成 LRU 策略。

(6) 测试结果

LRU 替换器测试结果如下：

```

gyh@ubuntu:~/winshare/rucbase-lab/build$ make lru_replacer_test
Scanning dependencies of target gtest_main
[ 62%] Building CXX object deps/googletest/googletest/CMakeFiles/gtest_main.dir/src/gtest_main.cc.o
[ 75%] Linking CXX static library ../../lib/libgtest_main.a
[ 75%] Built target gtest_main
Scanning dependencies of target lru_replacer_test
[ 87%] Building CXX object src/test/CMakeFiles/lru_replacer_test.dir/storage/lru_replacer_test.cpp.o
[100%] Linking CXX executable ../../bin/lru_replacer_test
[100%] Built target lru_replacer_test
● gyh@ubuntu:~/winshare/rucbase-lab/build$ ./bin/lru_replacer_test
Running main() from /mnt/hgfs/LAB/rucbase-lab/deps/googletest/googletest/src/gtest_main.cc
[=====] Running 3 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 3 tests from LRURewriterTest
[ RUN      ] LRURewriterTest.SimpleTest
[       OK ] LRURewriterTest.SimpleTest (0 ms)
[ RUN      ] LRURewriterTest.MixTest
[       OK ] LRURewriterTest.MixTest (9 ms)
[ RUN      ] LRURewriterTest.ConcurrencyTest
[       OK ] LRURewriterTest.ConcurrencyTest (66 ms)
[-----] 3 tests from LRURewriterTest (76 ms total)

[-----] Global test environment tear-down
[=====] 3 tests from 1 test suite ran. (76 ms total)
[ PASSED ] 3 tests.

```

(7) 实验结论

本任务实现了 LRU 页面替换策略，能够完成可淘汰 frame 的加入、移除和牺牲选择。通过该任务，我理解了 `pin`、`unpin` 与页面淘汰之间的关系，也理解了为什么 LRU 替换器只管理 `pin_count` 为 0 的页面。

任务2：缓冲池管理器实现

(1) 代码分析

本任务补全 `BufferPoolManager`，实现数据库页面在内存缓冲池和磁盘文件之间的调度。数据库访问页面时，先在缓冲池中查找；若不存在，则从磁盘读入；若缓冲池已满，则调用替换器淘汰一个可用 frame。

核心成员变量如下：

- `pool_size_`：缓冲池 frame 总数。
- `pages_`：缓冲池中的 Page 数组，页面实际数据存储在这里。
- `page_table_`：PageId -> frame_id 的映射，用来判断某个页面是否在缓冲池中。
- `free_list_`：空闲 frame 列表。优先使用空闲 frame，只有空闲列表为空时才使用 LRU 替换器。
- `replacer_`：页面替换器，从可淘汰 frame 中选出 victim。
- `disk_manager_`：磁盘管理器，负责 `read_page`、`write_page`、`allocate_page` 等底层操作。

Page 中的 `page_id_` 表示当前 frame 缓存的是哪个磁盘页，`pin_count_` 表示有多少上层操作正在使用该页，`is_dirty_` 表示该页在内存中是否被修改过。脏页被淘汰前必须写回磁盘，否则修改会丢失。

(2) `BufferPoolManager::find_victim_page` 函数的实现

该函数负责寻找一个可用 frame。实现时优先从 `free_list_` 中取空闲 frame；如果没有空闲 frame，再调用 `replacer_>victim` 使用 LRU 策略淘汰一个 frame。

```
bool BufferPoolManager::find_victim_page(frame_id_t* frame_id) {
    if (!free_list_.empty()) {
        *frame_id = free_list_.front();
        free_list_.pop_front();
        return true;
    }
    return replacer_->victim(frame_id);
}
```

这样设计可以减少不必要的磁盘写回。空闲 frame 中没有有效旧页，可以直接使用；只有无空闲 frame 时才需要走淘汰流程。

(3) BufferPoolManager::fetch_page 函数的实现

`fetch_page` 用于获取指定 `PageId` 的页面。若页面已经在缓冲池中，直接增加 `pin_count` 并从 LRU 中移除；若不在缓冲池中，则找一个 victim frame，必要时写回旧脏页，再从磁盘读取目标页。

实现流程如下：

1. 在 `page_table_` 中查找目标 `page_id`。
2. 如果命中，说明页面已经在内存中，将 `pin_count` 加一，并调用 `replacer_->pin(frame_id)` 防止被淘汰。
3. 如果未命中，调用 `find_victim_page` 获取可用 frame。
4. 如果 victim frame 原来缓存有效页面且为脏页，则调用 `disk_manager_->write_page` 写回磁盘。
5. 从 `page_table_` 中删除旧页面映射。
6. 调用 `disk_manager_->read_page` 将目标页面读入该 frame。
7. 更新页面元数据和页表，设置 `pin_count=1`、`is_dirty=false`。

(4) BufferPoolManager::unpin_page 函数的实现

`unpin_page` 表示上层使用完页面。函数先检查页面是否存在于缓冲池中，再减少 `pin_count`。如果 `pin_count` 降为 0，说明没有操作正在使用该页，可以调用 `replacer_->unpin(frame_id)` 将其加入可淘汰集合。

如果传入参数 `is_dirty` 为真，需要把页面脏位设为真，表示该页之后淘汰前必须写回。

(5) BufferPoolManager::flush_page 与 flush_all_pages 函数的实现

`flush_page` 将指定页面写回磁盘，并清除脏位。如果该页不在缓冲池中，则返回 `false`。

`flush_all_pages` 遍历缓冲池中的所有有效页面，将它们逐一写回磁盘。实现时不通过重复调用可能再次加锁的 `flush_page`，避免出现死锁风险。

(6) BufferPoolManager::new_page 函数的实现

`new_page` 用于在指定文件中分配一个新页。函数先寻找可用 frame，再调用 `disk_manager_->allocate_page(fd)` 分配新页号。如果 victim frame 是脏页，则先写回旧页。随后重置 frame 数据，设置新页面的 `PageId`、`pin_count=1`、`is_dirty=false`，并更新 `page_table_`。

新页刚被返回给上层时处于 pinned 状态，防止上层尚未完成初始化时被淘汰。

(7) BufferPoolManager::delete_page 函数的实现

delete_page 用于删除页面。如果页面不在缓冲池中，可以直接返回 true；如果页面在缓冲池中但 pin_count 不为 0，说明仍在被使用，不能删除，返回 false。只有当页面在缓冲池中且 pin_count 为 0 时，才从页表中删除映射、重置页面元数据，并将 frame 加回 free_list_。

(8) 自己声明的函数

本任务没有额外声明新的公开函数。实现主要依赖已有的 DiskManager、Page、LRUReplacer 接口共同完成。

(9) 测试结果

缓冲池管理器测试结果如下：

```
gyh@ubuntu:~/winshare/rucbase-lab/build$ make buffer_pool_manager_test
[ 60%] Built target storage
[ 80%] Built target gtest_main
Scanning dependencies of target buffer_pool_manager_test
[ 90%] Building CXX object src/test/CMakeFiles/buffer_pool_manager_test.dir/storage/buffer_pool_manager_test.cpp.o
[100%] Linking CXX executable ../../bin/buffer_pool_manager_test
[100%] Built target buffer_pool_manager_test
gyh@ubuntu:~/winshare/rucbase-lab/build$ ./bin/buffer_pool_manager_test
Running main() from /mnt/hgfs/LAB/rucbase-lab/deps/googletest/googletest/src/gtest_main.cc
[=====] Running 4 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 4 tests from BufferPoolManagerTest
[ RUN     ] BufferPoolManagerTest.SimpleTest
[ OK      ] BufferPoolManagerTest.SimpleTest (19 ms)
[ RUN     ] BufferPoolManagerTest.LargeScaleTest
[ OK      ] BufferPoolManagerTest.LargeScaleTest (8755 ms)
[ RUN     ] BufferPoolManagerTest.MultipleFilesTest
[ OK      ] BufferPoolManagerTest.MultipleFilesTest (3039 ms)
[ RUN     ] BufferPoolManagerTest.ConcurrencyTest
[ OK      ] BufferPoolManagerTest.ConcurrencyTest (60645 ms)
[-----] 4 tests from BufferPoolManagerTest (72459 ms total)

[-----] Global test environment tear-down
[=====] 4 tests from 1 test suite ran. (72459 ms total)
[ PASSED ] 4 tests.
```

(10) 实验结论

本任务实现了页面获取、新建、释放、刷新、删除和全部刷盘逻辑。通过本实验，我理解了缓冲池作为数据库内存与磁盘之间缓存层的作用，也理解了页表、空闲链表、LRU 替换器和脏页写回机制之间的协同关系。

实验2：记录管理器实现

任务1：记录操作实现

(1) 代码分析

本任务补全 RmFileHandle，实现表文件中定长记录的获取、插入、删除和更新。Record Manager 建立在 Lab1 的缓冲池之上，不直接读写磁盘，而是通过 BufferPoolManager 获取页面，再在页面内部管理记录槽位。

记录管理模块的核心结构如下：

- **Rid**：记录位置，由 **page_no** 和 **slot_no** 组成。**page_no** 定位数据页，**slot_no** 定位页内槽位。
- **RmFileHdr**：记录文件头，保存 **record_size**、**num_pages**、**num_records_per_page**、**first_free_page_no**、**bitmap_size** 等整张表文件的布局信息。
- **RmPageHdr**：数据页页头，保存当前页已有记录数和下一空闲页页号。
- **bitmap**：页内位图，每一位对应一个 slot。位为 1 表示该槽有有效记录，位为 0 表示该槽为空。
- **slot**：真正保存记录字节的位置。由于本实验是定长记录，某个槽地址可以通过 $slots + slot_no * record_size$ 计算得到。

空闲页链表由 **file_hdr.first_free_page_no** 和各页 **page_hdr->next_free_page_no** 维护。插入时优先从空闲页链表找页；页面插满后从链表移除；满页删除记录后重新加入链表。

(2) **RmFileHandle::get_record** 函数的实现

get_record 根据 **Rid** 读取指定记录。函数先通过 **rid.page_no** 获取页面，再检查 **rid.slot_no** 是否越界以及 **bitmap** 对应位是否为 1。若该槽没有有效记录，则抛出 **RecordNotFoundError**。

实现逻辑如下：

1. 调用 **fetch_page_handle(rid.page_no)** 获取页面句柄。
2. 检查 **slot_no** 合法性和 **bitmap** 位。
3. 通过 **page_handle.get_slot(rid.slot_no)** 得到槽起始地址。
4. 创建 **RmRecord**，拷贝 **record_size** 字节数据。
5. 读操作不修改页面，调用 **unpin_page(..., false)**。

这样返回的 **RmRecord** 拥有独立内存，即使页面之后被 **unpin** 或淘汰，记录内容仍然有效。

(3) **RmFileHandle::insert_record** 函数的实现

未指定位置的插入函数负责自动寻找一个空槽。实现时先调用 **create_page_handle** 获取有空闲槽的页面，再通过 **bitmap** 找到第一个 0 位，将记录字节拷贝到对应 slot。

插入后需要完成以下维护：

- 将 **bitmap** 对应位置为 1。
- 页内 **num_records** 加一。
- 构造并返回新记录的 **Rid**。
- 如果页面插入后变满，则从空闲页链表移除该页。
- 页面内容和页头被修改，因此 **unpin** 时 **dirty** 置为 **true**。

指定位置插入版本主要供恢复或后续模块使用。它根据传入的 **Rid** 定位页面和槽位，如果原槽位为空，则设置 **bitmap** 并增加页内记录数；如果插入后页面变满，也需要从空闲页链表中摘除该页。

(4) **RmFileHandle::delete_record** 函数的实现

delete_record 根据 **Rid** 删除记录。删除并不要求清空 slot 中旧字节，因为系统判断记录是否有效依赖 **bitmap**，而不是 slot 中是否残留数据。

实现流程如下：

1. 获取目标页面。
2. 检查目标 slot 是否有效。
3. 记录删除前页面是否已满。
4. 将 bitmap 对应位清 0，页内 `num_records` 减一。
5. 如果该页删除前是满页、删除后变为未满，则调用 `release_page_handle` 将其加入空闲页链表。
6. 修改了 bitmap 和页头，unpin 时 dirty 置为 true。

(5) `RmFileHandle::update_record` 函数的实现

`update_record` 根据 `Rid` 找到指定槽位，并用传入缓冲区覆盖原记录内容。更新前同样需要检查 slot 是否有效。因为记录是定长的，所以可以直接 `memcpy record_size` 字节到目标槽位。

更新只改变记录内容，不改变 bitmap、记录数量和空闲页链表。

(6) 页面辅助函数的实现

`fetch_page_handle` 调用缓冲池的 `fetch_page` 获取指定页面，并封装为 `RmPageHandle`，方便访问页头、bitmap 和 slots。

`create_new_page_handle` 用于创建新的记录页。它通过 `buffer_pool_manager->new_page` 申请新页，初始化页头，将 `num_records` 设为 0，并把该页加入空闲页链表，同时更新文件头 `num_pages`。

`create_page_handle` 先检查 `file_hdr_.first_free_page_no`。如果存在空闲页，就取该页；如果不存在，则调用 `create_new_page_handle` 新建页。

`release_page_handle` 用于在满页删除记录后，把该页重新挂到空闲页链表头部，使之后插入可以复用释放出的 slot。

(7) 自己声明的函数

本任务没有新增公开函数。实现中主要补全指导书要求的 `RmFileHandle` 成员函数，并使用已有的 `Bitmap` 工具函数维护页内槽位状态。

(8) 测试结果

记录操作测试结果如下：


```

gyh@ubuntu:~/winshare/rucbase-lab/build$ make record_manager_test
[ 70%] Built target system
[ 83%] Built target record
[ 91%] Built target gtest_main
Scanning dependencies of target record_manager_test
[ 95%] Building CXX object src/test/CMakeFiles/record_manager_test.dir/storage/record_manager_test.cpp.o
[100%] Linking CXX executable ../../bin/record_manager_test
[100%] Built target record_manager_test
gyh@ubuntu:~/winshare/rucbase-lab/build$ ./bin/record_manager_test
Running main() from /mnt/hgfs/LAB/rucbase-lab/deps/googletest/googletest/src/gtest_main.cc
[=====] Running 2 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 2 tests from RecordManagerTest
[ RUN      ] RecordManagerTest.SimpleTest
insert 452
delete 291
update 257
[ OK      ] RecordManagerTest.SimpleTest (397 ms)
[ RUN      ] RecordManagerTest.MultipleFilesTest
[ OK      ] RecordManagerTest.MultipleFilesTest (309 ms)
[-----] 2 tests from RecordManagerTest (707 ms total)

[-----] Global test environment tear-down
[=====] 2 tests from 1 test suite ran. (707 ms total)
[ PASSED  ] 2 tests.

```

(9) 实验结论

本任务实现了按 **Rid** 获取记录、自动插入记录、指定位置插入记录、删除记录和更新记录。通过本任务，我理解了定长记录文件中 page header、bitmap 和 slots 三部分的组织方式，也理解了空闲页链表在提升插入效率中的作用。

任务2：记录迭代器实现

(1) 代码分析

本任务补全 **RmScan**，实现对表文件中所有有效记录的顺序遍历。**RmScan** 内部保存当前 **rid₀**，通过 bitmap 找到下一个值为 1 的 slot。它只负责定位记录位置，不负责解释记录字段内容。

RmScan 继承自 **RecScan**，主要接口包括：

- **next()**：移动到下一条有效记录。
- **is_end()**：判断扫描是否结束。
- **rid()**：返回当前记录位置。

(2) **RmScan::RmScan** 构造函数的实现

构造函数接收 **RmFileHandle***，初始化扫描器所属的文件句柄，并把初始 **rid₀** 设在扫描起点之前，再调用 **next()** 定位到第一条有效记录。

这样设计后，外部创建 **RmScan** 后即可直接读取当前 **rid()**，无需额外手动寻找第一条记录。

(3) **RmScan::next** 函数的实现

next 的目标是从当前 **rid₀** 之后开始寻找下一条有效记录。实现中按页遍历，每页通过 **Bitmap::next_bit(true, ...)** 查找下一个 bitmap 为 1 的槽位。

如果当前页找到有效 slot，则更新 `rid_.slot_no` 并返回；如果当前页没有有效记录，则 unpin 当前页，进入下一页继续找。当所有页都扫描完后，将 `rid_.page_no` 设为 `RM_NO_PAGE`，表示扫描结束。

每次通过缓冲池获取页面后，都必须在使用完后 unpin。扫描只读 bitmap，不修改页面，因此 dirty 参数为 false。

(4) `RmScan::is_end` 函数的实现

`is_end` 判断当前扫描是否结束。本实现中使用 `rid_.page_no == RM_NO_PAGE` 或 `rid_.page_no >= file_hdr_.num_pages` 作为结束标志。当扫描器结束时，上层执行器不应再调用 `get_record` 读取当前记录。

(5) 测试结果

记录迭代器与记录操作共用同一单元测试，测试结果如下：

```
gyh@ubuntu:~/winshare/rucbase-lab/build$ make record_manager_test
[ 70%] Built target system
[ 83%] Built target record
[ 91%] Built target gtest_main
Scanning dependencies of target record_manager_test
[ 95%] Building CXX object src/test/CMakeFiles/record_manager_test.dir/storage/record_manager_test.cpp.o
[100%] Linking CXX executable ../../bin/record_manager_test
[100%] Built target record_manager_test
gyh@ubuntu:~/winshare/rucbase-lab/build$ ./bin/record_manager_test
Running main() from /mnt/hgfs/LAB/rucbase-lab/deps/googletest/googletest/src/gtest_main.cc
[=====] Running 2 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 2 tests from RecordManagerTest
[ RUN      ] RecordManagerTest.SimpleTest
insert 452
delete 291
update 257
[ OK      ] RecordManagerTest.SimpleTest (397 ms)
[ RUN      ] RecordManagerTest.MultipleFilesTest
[ OK      ] RecordManagerTest.MultipleFilesTest (309 ms)
[-----] 2 tests from RecordManagerTest (707 ms total)

[-----] Global test environment tear-down
[=====] 2 tests from 1 test suite ran. (707 ms total)
[ PASSED  ] 2 tests.
```

(6) 实验结论

本任务实现了基于 bitmap 的记录扫描器。通过该任务，我理解了表扫描并不是直接读取所有数据，而是逐页查看 bitmap，只返回真正有效的记录位置。这也为实验四中的顺序扫描执行器提供了基础。

实验3：数据定义的实现

任务1：元数据管理实现

(1) 代码分析

本实验要求补全 `SmManager` 类，使 Rucbase 支持基本数据定义功能，包括创建表、删除表、显示表、描述表以及数据库打开和关闭时的元数据管理。本次实现重点为 `open_db`、`close_db` 和 `drop_table`。

系统管理模块的核心数据结构如下：

- **DbMeta**: 数据库元数据, 保存数据库名和所有表的元数据映射 **tabs_**。
- **TabMeta**: 表元数据, 保存表名、列信息 **cols** 和索引信息 **indexes**。
- **ColMeta**: 列元数据, 保存列名、类型、长度、偏移量 **offset** 和是否有索引。
- **fhs_**: 表名到 **RmFileHandle** 的映射, SQL 执行时通过它读写表记录。
- **ih_**: 索引文件名到 **IxIndexHandle** 的映射, 后续索引模块通过它读写索引。

数据库在文件系统中表现为一个同名目录, 目录中包含 **db.meta**、表文件、索引文件和日志文件。**db.meta** 只保存元数据, 不保存真实记录。真实记录由记录管理器管理。

(2) **SmManager::open_db** 函数的实现

open_db 的作用是打开数据库目录, 读取 **db.meta**, 恢复内存中的数据库元数据, 并打开所有表文件和索引文件。

实现流程如下:

1. 判断数据库同名目录是否存在, 不存在则抛出 **DatabaseNotFoundError**。
2. 调用 **chdir(db_name.c_str())** 进入数据库目录。
3. 清空内存中的旧元数据和旧文件句柄, 避免上一次打开的数据库状态残留。
4. 打开 **db.meta**, 使用重载的 **operator>>** 将文件中的元数据读入 **db_**。
5. 遍历 **db_.tabs_**, 对每张表调用 **rm_manager_->open_file(tab_name)**, 并保存到 **fhs_**。
6. 对每张表已有的索引, 调用 **ix_manager_->open_index** 打开索引文件, 并保存到 **ih_**。

核心代码结构如下:

```
void SmManager::open_db(const std::string& db_name) {
    if (!is_dir(db_name)) {
        throw DatabaseNotFoundError(db_name);
    }
    if (chdir(db_name.c_str()) < 0) {
        throw UnixError();
    }

    db_ = DbMeta();
    fhs_.clear();
    ih_.clear();

    std::ifstream ifs(DB_META_NAME);
    if (!ifs.is_open()) {
        throw FileNotFoundError(DB_META_NAME);
    }
    ifs >> db_;

    for (auto &entry : db_.tabs_) {
        const std::string &tab_name = entry.first;
        TabMeta &tab = entry.second;
        fhs_.emplace(tab_name, rm_manager_->open_file(tab_name));
        for (auto &index : tab.indexes) {
            std::string index_name = ix_manager_->get_index_name(tab_name,
index.cols);
            ih_.emplace(index_name, ix_manager_->open_index(tab_name,
```

```
index.cols));  
    }  
}  
}
```

(3) `SmManager::close_db` 函数的实现

`close_db` 的作用是关闭当前数据库并把元数据、表文件和索引文件的状态写回磁盘。

实现流程如下：

1. 调用 `flush_meta()` 将当前 `db_` 写回 `db.meta`。
2. 遍历 `ihs_`，调用 `ix_manager_->close_index` 关闭所有打开的索引文件。
3. 清空 `ihs_`。
4. 遍历 `fhs_`，调用 `rm_manager_->close_file` 关闭所有打开的表文件。
5. 清空 `fhs_`。
6. 调用 `chdir("../")` 回到上一级目录。

其中，表文件头中保存记录长度、页数、空闲页链表等信息；索引文件头中保存索引结构信息。关闭时写回这些信息，才能保证下次打开数据库后状态一致。

(4) `SmManager::drop_table` 函数的实现

`drop_table` 用于删除一张表。删除表时不能只删除表数据文件，还必须同步删除该表上的索引文件，并更新 `db.meta`。

实现流程如下：

1. 判断表是否存在，不存在则抛出 `TableNotFoundError`。
2. 通过引用获取 `TabMeta &tab`。这里必须使用引用，因为 `TabMeta` 的拷贝可能不会完整保留索引信息。
3. 遍历该表的所有索引：如果索引句柄在 `ihs_` 中，先关闭并从 `ihs_` 移除；然后调用 `ix_manager_->destroy_index` 删除索引文件。
4. 在 `fhs_` 中查找表文件句柄，如果存在则先关闭并移除。
5. 调用 `rm_manager_->destroy_file(tab_name)` 删除表文件。
6. 从 `db_.tabs_` 中删除该表元数据。
7. 调用 `flush_meta()` 将元数据变化写回 `db.meta`。

关键点是“先关闭再删除”。如果文件仍处于打开状态，直接删除可能导致底层文件管理器认为文件仍被占用。

(5) 自己声明的函数

本任务没有额外声明新的公开函数，主要补全指导书指定的 `SmManager` 成员函数。实现中复用了已有的 `is_dir`、`flush_meta`、`RmManager` 和 `IxManager` 接口。

(6) 测试结果

实验三测试结果如下：

```

gyh@ubuntu:~/rucbase-lab-test/src/test/query$ python3 query_unit_test.py basic_query_test1.sql
Waiting for request...
i_recvBytes: 60
Read from client 4: -- 删除下面3条语句的原因是跳过了索引实验
Waiting for request...
i_recvBytes: 28
Read from client 4: -- create index grade(id);
Waiting for request...
i_recvBytes: 36
Read from client 4: -- create index grade(student_id);
Waiting for request...
i_recvBytes: 30
Read from client 4: -- create index student(id);
Waiting for request...
i_recvBytes: 2
Read from client 4:
Waiting for request...
i_recvBytes: 19
Read from client 4: drop table grade;
Waiting for request...
Maybe the client has closed
Terminating current client_connection...
finish kill
finish delete database
Unit Test Score: 25.0

```

测试中出现的 `Table already exists: grade` 和 `Table not found: t` 是测试脚本故意触发的异常，用于验证重复建表和删除不存在表时的错误处理是否正确。最终 `Unit Test Score: 25.0` 表示该测试点通过。

(7) 实验结论

本实验实现了数据库打开、关闭和删除表的元数据管理逻辑。通过本实验，我理解了 `db.meta` 与内存元数据 `db_` 的关系，也理解了 `fhs_`、`ihs_` 作为运行时文件句柄表在 SQL 执行过程中的作用。

实验4：数据操纵的实现

任务1：顺序扫描算子的实现

(1) 代码分析

本任务补全 `SeqScanExecutor`，实现顺序扫描算子。顺序扫描是查询执行树的叶子节点，负责从一张表中逐条产生满足 `where` 条件的记录。

所有执行器都继承 `AbstractExecutor`，采用类似 Volcano 的迭代器模型：

- `beginTuple()`：定位到第一条结果。
- `Next()`：返回当前结果，但不移动。
- `nextTuple()`：移动到下一条结果。
- `is_end()`：判断是否结束。

`SeqScanExecutor` 的核心成员如下：

- `tab_name_`：被扫描的表名。
- `conds_ / fed_conds_`：选择条件。
- `fh_`：表文件句柄，用于按 `Rid` 读取记录。
- `cols_`：输出记录的列元数据。
- `len_`：一条输出记录的字节长度。
- `rid_`：当前满足条件的记录位置。

- `scan_`: 底层 `RmScan` 记录迭代器。

其中 `rid_` 保存的是当前结果记录的位置，不是记录内容。真正返回记录时需要调用 `fh_->get_record(rid_, context_)`。

(2) `SeqScanExecutor::beginTuple` 函数的实现

`beginTuple` 创建 `RmScan`，从表的第一条有效记录开始扫描，直到找到第一条满足条件的记录。

实现流程如下：

1. 创建 `scan_ = std::make_unique<RmScan>(fh_)`。
2. 循环判断 `scan_->is_end()`。
3. 调用 `fh_->get_record(scan_->rid(), context_)` 读取当前候选记录。
4. 调用 `eval_conds` 判断是否满足所有条件。
5. 若满足，则将当前 `scan_->rid()` 保存到 `rid_` 并返回。
6. 若扫描结束仍未找到，设置 `rid_ = {RM_NO_PAGE, -1}` 表示结束。

(3) `SeqScanExecutor::nextTuple` 函数的实现

`nextTuple` 从当前记录之后继续扫描，寻找下一条满足条件的记录。因为当前 `rid_` 已经作为上一条结果返回，所以函数开始时先调用 `scan_->next()`，再重复判断条件。

如果没有下一条满足条件的记录，同样将 `rid_` 设置为结束标志。

(4) `SeqScanExecutor::Next` 函数的实现

`Next` 返回当前 `rid_` 对应的记录。如果执行器已经结束，返回 `nullptr`；否则调用 `fh_->get_record(rid_, context_)` 获取记录。

条件判断时，列值通过 `rec->data + col.offset` 定位。对于常量右值，语义分析阶段已经将其转换为 `raw` 字节；对于列和列比较，则通过右侧列的 `offset` 从同一条记录中取值。

(5) 自己声明的函数

为了完成条件判断，我实现了辅助函数：

- `compare_value`: 按列类型比较两个字段的原始字节。
- `compare_result`: 根据比较结果和操作符判断条件真假。
- `eval_cond`: 判断单个条件是否成立。
- `eval_conds`: 判断所有条件是否都成立。

这些函数都是类内私有辅助函数，不改变执行器的公开接口。

(6) 测试结果

顺序扫描功能包含在实验四完整测试中，测试结果见实验四末尾完整测试截图。

(7) 实验结论

本任务实现了基于 `RmScan` 的顺序扫描算子，能够逐条遍历表记录并筛选满足条件的记录。通过本任务，我理解了记录管理器和查询执行器之间的衔接关系。

任务2：投影算子的实现

(1) 代码分析

本任务补全 `ProjectionExecutor`，实现 `select` 后指定列的输出。投影算子本身不扫描表，而是从子执行器 `prev_` 获取完整记录，再拷贝需要的列形成新的结果记录。

核心成员如下：

- `prev_`：子执行器，可能是顺序扫描，也可能是 join 结果。
- `cols_`：投影后输出列的元数据。
- `len_`：投影后一条结果记录的总长度。
- `sel_idx_`：每个输出列在 `prev_->cols()` 中的下标。

`prev_` 的作用是向下获取完整记录。投影算子只负责列裁剪，不负责判断条件或扫描表。

(2) 构造函数和 `offset` 重算

构造函数根据 `sel_cols` 找到每个被选列在子执行器输出列中的位置，并保存到 `sel_idx_`。同时，投影后的记录布局已经不同于原始记录，因此每个输出列的 `offset` 必须从 0 重新累加。

例如原记录为 `id, name, age`，执行 `select name, id` 后，输出记录中 `name.offset = 0`，`id.offset = name.len`，不能再使用原表中的偏移量。

(3) `beginTuple` 和 `nextTuple` 函数的实现

投影算子与子执行器是一一对应关系，所以：

- `beginTuple()` 直接调用 `prev_->beginTuple()`。
- `nextTuple()` 直接调用 `prev_->nextTuple()`。
- `is_end()` 直接返回 `prev_->is_end()`。

(4) `ProjectionExecutor::Next` 函数的实现

`Next` 先调用 `prev_->Next()` 取子节点当前完整记录，再创建长度为 `len_` 的新 `RmRecord`。随后遍历 `sel_idx_`，根据源列 `src_col.offset` 从子记录中取字段，根据目标列 `dst_col.offset` 写入投影结果。

核心拷贝逻辑如下：

```
memcpy(projected->data + dst_col.offset,
        prev_rec->data + src_col.offset,
        src_col.len);
```

这样只复制被选择的列，不会把未选择的字段带入结果。

(5) 测试结果

投影算子测试结果如下：

```
gyh@ubuntu:~/rucbase-lab-test/src/test/query$ python3 query_unit_test.py basic_query_test2.sql
Read from client 4: select major from student where name = 'JerryJerr';
Waiting for request...
i_recvBytes: 69
Read from client 4: select name,major from student where name = 'JerryJerr' and id = 0;
Waiting for request...
i_recvBytes: 69
Read from client 4: select name,major from student where name = 'JerryJerr' and id = 2;
Waiting for request...
i_recvBytes: 79
Read from client 4: select name,id from student where major = 'aaaaaaaaaaaaaaaaaaaaaaaaaaaaa';
Waiting for request...
i_recvBytes: 29
Read from client 4: select myname from student;
Error: Column not found: myname
Waiting for request...
i_recvBytes: 48
Read from client 4: select name from student where myname = 'aaa';
Error: Column not found: myname
Waiting for request...
Maybe the client has closed
Terminating current client_connection...
finish kill
finish delete database
Unit Test Score: 15.0
```

(6) 实验结论

本任务实现了投影算子，理解了查询输出记录的布局可能与原表记录不同，因此必须重新计算输出列的 `offset`。

任务3：嵌套循环连接算子的实现

(1) 代码分析

本任务补全 `NestedLoopJoinExecutor`，实现基础的嵌套循环连接。该算子有左右两个子执行器：`left_` 是外层循环，`right_` 是内层循环。对每一条左记录，都从头遍历右记录，满足 join 条件时输出一条拼接记录。

核心成员如下：

- `left_`：左子执行器。
- `right_`：右子执行器。
- `len_`：连接后记录长度，等于左记录长度加右记录长度。
- `cols_`：连接后输出列元数据。
- `fed_conds_`：连接条件。
- `isend`：连接算子是否结束。

连接后记录的内存布局为“左记录字节 + 右记录字节”。因此右表字段的 `offset` 要整体加上 `left_>tupleLen()`。

(2) `beginTuple` 函数的实现

`beginTuple` 先让左子执行器定位到第一条记录。如果左输入为空，则 `join` 直接结束。否则再让右子执行器定位到第一条记录，并调用 `find_next_match()` 寻找第一对满足连接条件的记录。

(3) `nextTuple` 函数的实现

当前组合被上层取走后，`nextTuple` 先移动右子执行器到下一条记录，然后调用 `find_next_match()` 继续寻找下一对匹配记录。如果右表扫描结束，`find_next_match()` 会推动左表前进一条，并重新调用 `right_>beginTuple()` 从右表开头继续匹配。

(4) `Next` 函数的实现

`Next` 返回当前左右记录拼接后的结果。它分别调用 `left_>Next()` 和 `right_>Next()` 获取当前左右记录，然后创建长度为 `len_` 的新记录，将左记录复制到前半部分，将右记录复制到后半部分。

```
memcpy(joined->data, left_rec->data, left_>tupleLen());
memcpy(joined->data + left_>tupleLen(), right_rec->data, right_>tupleLen());
```

(5) 自己声明的函数

我实现了以下私有辅助函数：

- `find_next_match`：推动左右子执行器，直到找到下一对满足条件的记录。
- `value_ptr`：根据连接后列的 `offset` 判断字段来自左记录还是右记录。
- `compare_value`、`compare_result`、`eval_cond`、`eval_conds`：完成 `join` 条件比较。

当 `fed_conds_` 为空时，`eval_conds` 返回 `true`，对应没有连接条件时的笛卡尔积。

(6) 测试结果

嵌套循环连接测试结果如下：

```
gyh@ubuntu:~/rucbase-lab-test/src/test/query$ python3 query_unit_test.py basic_query_test5.sql
Read from client 4: insert into grade values ('DataDatad', 3, 97.5);
Waiting for request...
i_recvBytes: 50
Read from client 4: insert into grade values ('DataDataa', 4, 82.0);
Waiting for request...
i_recvBytes: 50
Read from client 4: insert into grade values ('DataDatab', 4, 85.0);
Waiting for request...
i_recvBytes: 50
Read from client 4: insert into grade values ('DataDataac', 4, 85.0);
Waiting for request...
i_recvBytes: 50
Read from client 4: insert into grade values ('DataDatad', 4, 90.5);
Waiting for request...
i_recvBytes: 31
Read from client 4: select * from student, grade;
Waiting for request...
i_recvBytes: 96
Read from client 4: select id, name, major, course, score from student, grade where student.id = grade.student_id;
Waiting for request...
Maybe the client has closed
Terminating current client_connection...
finish kill
Unit Test Score: 30.0
```

(7) 实验结论

本任务实现了最基础的 join 算子。通过该任务，我理解了连接结果的字节布局、左右记录 offset 的转换方式，以及嵌套循环连接的执行流程。

任务4：修改算子的实现

(1) 代码分析

本任务补全 `UpdateExecutor`，实现 `update` 语句。项目中 `Portal` 会先通过查询条件扫描出所有需要更新记录的 `Rid`，保存到 `rids_`，因此 `UpdateExecutor` 不需要重新判断 `where` 条件。

核心成员如下：

- `tab_`：被更新表的元数据，用于查字段类型、长度和 offset。
- `fh_`：表文件句柄，用于读取旧记录和写回新记录。
- `rids_`：所有需要更新的记录位置。
- `set_clauses_`：`set` 子句列表。

`rids_` 是由上层 `Portal` 在构造更新执行器之前，通过扫描执行计划收集得到的。

(2) `UpdateExecutor::Next` 函数的实现

`Next` 遍历 `rids_` 中的所有记录位置。对每个 `Rid`，先调用 `fh_->get_record` 读取旧记录，再复制出一条 `new_record`。复制旧记录的原因是只覆盖 `set` 指定的字段，未被修改的字段应保持原值。

对每个 `set_clause`，函数根据左侧列名找到 `ColMeta`，检查右值类型是否与列类型一致。如果 `rhs.raw` 尚未初始化，则调用 `init_raw(col->len)` 转换为字节形式。最后通过 `memcpy(new_record.data + col->offset, ...)` 覆盖目标字段。

所有字段修改完成后，调用：

```
fh_->update_record(rid, new_record.data, context_);
```

将新记录写回原位置。

(3) 测试结果

修改算子测试结果如下：

```
gyh@ubuntu:~/rucbase-lab-test/src/test/query$ python3 query_unit_test.py basic_query_test3.sql
i_recvBytes: 56
  Read from client 4: update student set id = 100 where name = 'JerryJerry';
Error: String is too long
Waiting for request...
i_recvBytes: 37
  Read from client 4: select * from student where id > 2;
Waiting for request...
i_recvBytes: 39
  Read from client 4: select * from student where id < 101;
Waiting for request...
i_recvBytes: 98
  Read from client 4: update student set name = '789123456' , major = 'bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb' where id < 3;
Waiting for request...
i_recvBytes: 37
  Read from client 4: select * from student where id = 2;
Waiting for request...
i_recvBytes: 49
  Read from client 4: select * from student where name = '789123456';
Waiting for request...
Maybe the client has closed
Terminating current client_connection...
finish kill
finish delete database
Unit Test Score: 15.0
```

(4) 实验结论

本任务实现了按 **Rid** 批量更新记录的逻辑。通过本任务，我理解了 **update** 并不是直接修改整张表，而是先确定要更新的记录位置，再逐条修改指定字段。

任务5：删除算子的实现

(1) 代码分析

本任务补全 **DeleteExecutor**，实现 **delete** 语句。与 **update** 类似，**Portal** 已经提前根据 **where** 条件收集所有需要删除记录的 **Rid**，删除执行器只需要遍历这些 **Rid** 并调用记录管理器接口。

核心成员如下：

- **tab_**：表元数据，当前基础删除逻辑中不直接使用，但后续维护索引删除时需要。
- **conds_**：delete 条件，真正筛选已经在构造 **rids_** 前完成。
- **fh_**：表文件句柄。
- **rids_**：需要删除的记录位置。

(2) **DeleteExecutor::Next** 函数的实现

Next 遍历 **rids_**，对每个 **Rid** 调用：

```
fh_ -> delete_record(rid, context_);
```

删除记录时并不直接在执行器中清理 slot，而是交给 Lab2 实现的 **RmFileHandle::delete_record**。该函数会清 bitmap、减少页内记录数，并在必要时把页面重新挂回空闲页链表。这样后续插入记录时可以复用删除释放出的 slot。

delete 是修改语句，不产生结果表，所以 **Next** 执行完成后返回 **nullptr**。

(3) 测试结果

删除算子测试结果如下：

```
gyh@ubuntu:~/rucbase-lab-test/src/test/query$ python3 query_unit_test.py basic_query_test4.sql
Waiting for request...
i_recvBytes: 58
  Read from client 4: delete from student where id = 3 and name = 'JackJackJ';
Waiting for request...
i_recvBytes: 24
  Read from client 4: select * from student;
Waiting for request...
i_recvBytes: 40
  Read from client 4: select name from student where id = 3;
Waiting for request...
i_recvBytes: 52
  Read from client 4: select name from student where name = 'JackJackJ';
Waiting for request...
i_recvBytes: 71
  Read from client 4: delete from student where major = 'aaaaaaaaaaaaaaaaaaaaaaaaaaaaa';
Waiting for request...
i_recvBytes: 36
  Read from client 4: select * from student where id>=1;
Waiting for request...
Maybe the client has closed
Terminating current client_connection...
finish kill
finish delete database
Unit Test Score: 15.0
```

实验四完整测试结果如下：

```
gyh@ubuntu:~/rucbase-lab-test/src/test/query$ python3 query_test_basic.py
Read from client 4: insert into grade values ('DataDatad', 3, 97.5);
Waiting for request...
i_recvBytes: 50
  Read from client 4: insert into grade values ('DataDataa', 4, 82.0);
Waiting for request...
i_recvBytes: 50
  Read from client 4: insert into grade values ('DataDatab', 4, 85.0);
Waiting for request...
i_recvBytes: 50
  Read from client 4: insert into grade values ('DataDataac', 4, 85.0);
Waiting for request...
i_recvBytes: 50
  Read from client 4: insert into grade values ('DataDatad', 4, 90.5);
Waiting for request...
i_recvBytes: 31
  Read from client 4: select * from student, grade;
Waiting for request...
i_recvBytes: 96
  Read from client 4: select id, name, major, course, score from student, grade where student.id = grade.student_id;
Waiting for request...
Maybe the client has closed
Terminating current client_connection...
finish kill
final score: 100.0
```

(4) 实验结论

本任务实现了按 **Rid** 删除记录的逻辑。结合 update 任务可以看出，DML 修改语句的执行流程是先通过查询执行器找出目标记录位置，再调用记录管理器完成实际修改。

总结

四个实验从底层到上层逐步实现了 Rucbase 的核心功能。实验一实现缓冲池与页面替换，解决“页面如何在内存和磁盘之间调度”的问题；实验二实现记录管理器，解决“一页中如何组织和访问记录”的问题；实验三实现元数据管理，解决“数据库、表、列和文件句柄如何对应”的问题；实验四实现查询与数据操纵执行器，解决“SQL 如何转化为逐条记录的执行过程”的问题。

通过这四个实验，我对数据库系统中存储管理、缓冲管理、元数据管理和查询执行之间的联系有了更完整的认识。